Backend Web Development with API Integration (CSC103)

Building Web Service using



Prepared by: Ms. Tshering Lhamo

Overview

This session covers the development environment setup for building Go application followed by PostgreSQL database server setup. You will learn how to establish a connection between the Go application and database server. Further, you will develop the REST API for student and course modules of the Enroll Application by implementing the MVC design pattern.

Contents

1. Building Frontend **3**
Serving Static Content 3
2. Connecting to Backend Using JavaScript 4
Handling the HTTP Response 6
Sign Up and Login 17
Handling indexes 19
3. Sign up and Login API **20**
4. Session Management **26**
Cookie 26
Session Management Using Cookie 29
Securing Web API 33
5. Enroll APIs **36**
6. Enroll View 41
7. Testing **48**
Writing Unit Test in Golang 48
8. Deployment **53**
Creating PostgreSQL Database 53
Connect to Render PostgreSQL using pgAdmin 55
Connecting Local Application to the Render PostgreSQL Database 57
Deploying Web Service 59
Reference **64**

1. Building Frontend

- Before moving to creating the remaining APIs, you will build the UI of the application and make use of the student and course API from the frontend.

1.1. Create a new directory **view** at the root directory of your project. This directory serves all the static files.
1.2. Inside the view create html files **student.html**, **course.html, enroll.html, signup.html** and **login.html**.
1.3. Inside the view create a new directory **css**. Inside the css directory, create new files **base.css** and **login.css** for defining styles.
1.4. Download the static files from here. Copy and paste the code in the respective files within your project.
1.5. Within the view directory, create a new directory **js** to store all the script files. Within the **js** directory create script files **student.js, course.js, enroll.js, login.js** and **signup.js**.

Serving Static Content

- Static files are files that do not change. They include CSS files, JavaScript files and images; also plain HTML files which do not contain template directives.

Using **HTTP file server** with **gorilla/mux**

- By default, an HTTP web server does not automatically serve static files such as HTML, CSS, JavaScript, or images from the server directory. Therefore, it is necessary to configure the server to handle requests for these static resources.

- In Go, the **http.FileServer()** function from the **net/http** package is used to serve static files. It returns an HTTP handler that responds to incoming requests by providing the corresponding files from a specified directory in the file system.

```
func FileServer(root FileSystem) Handler
```

- As you can see, it returns a **Handler object** and takes **FileSystem interface** type as input.

```
type FileSystem interface {
    Open(name string) (http.File, error)
}
```

- When using the Gorilla Mux router, this file server handler can be registered as a route to allow the application to serve static content. This enables the web server to deliver frontend resources such as stylesheets, scripts, and images to the client when requested.

1.6. Add the following two statements in the **routes.go**, right before you start the server.

```
router.HandleFunc("/students", controller.GetAllStu

// serve static files
fhandler := http.FileServer(http.Dir("./views"))
router.PathPrefix("/").Handler(fhandler)

log.Println("Application running on port", port)
```

`http.Dir type`

- The **Dir** type may look like a function but it is an alias of **string** data type and it implements **Open** method defined by the FileSystem interface. You can call the Dir type like a function which is nothing but a **type-casting** syntax.
- The **string value** passed to the type-casting syntax is a relative or an absolute directory on the native file-system.

```
var fs FileSystem = http.Dir("./view")
```

- Next, **router.PathPrefix("/")** is used to create a route that matches the request with the / prefix.
- ***router.Handler*** method registers the file server as the handler to handle requests that match the prefix "/".

2. Connecting to Backend Using JavaScript

Add a student detail to the database

- Use Post API to add a single student detail to the database.

2.1. In the student.html page, add **onclick** property to the **Add** button and assign value **addStudent().**

```
<div class="row">
    <button id="button-add" onclick="addStudent()">Add</button>
</div>
```

2.2. Open student.js file and create a function **addStudent()**. This function retrieves the form data submitted by the client and calls the /student API to store the student data.

```
function addStudent(){
}
```

2.3. Implement the function as shown below.

```
function addStudent(){
     var data = {
        stdid : parseInt(document.getElementById("sid").value),
        fname : document.getElementById("fname").value,
        lname : document.getElementById("lname").value,
        email : document.getElementById("email").value
    }
    fetch('/student', {
        method: "POST",
        body: JSON.stringify(data),
        headers: {"Content-type": "application/json; charset=UTF-8"}
    });
}
```

- In the above code **data** is the JavaScript object that stores the student data retrieved from the form UI.
- **fetch()** takes two arguments, the **url**("/student") and **request object**. Since, this is a POST request, you need to specify the method **explicitly** otherwise, fetch **defaults** to GET requests.
- The request object consists of request method, request body, and request header.
- *JSON.stringify()* converts JavaScript objects to JSON string type.

- In *student.html* import the script file right before the closing body tag.

```
<script src="js/student.js"></script>
```

- ➢ Now restart the server and launch the application in the browser.
- ➢ To test the addStudent function fill in the form details and click on the Add button to send a request to the server.
- ➢ Check the student table in the database.

Handling the HTTP Response

Using then method

- If the POST request is successful, send the GET request to retrieve a single student data which is added by the Post request and display it in the table.

Retrieve a single student data

2.4. Update the addStudent() by using then method to handle the HTTP response. In this method call, the API that retrieves the single student data and displays the data.

- ➢ Here is the code.

```
}).then(response1 => {
    var sid = data.stdid;
    if (response1.ok) {
        fetch('/student/'+sid)
        .then(response2 => response2.text())
        .then(data => showStudent(data))
    }
})
```

- ➢ The promise returned by fetch calls **then** method. The method body is implemented using an arrow function and the argument received by then is stored in a variable **response1** which is of type **response object.**
- ➢ In the method body it checks if the HTTP response status text is a **success (OK)** and if so, it uses fetch() to send a **GET** request to the API url **"*student/sid*"** with url parameter **sid.**
- ➢ If the promise returned by fetch is resolved, it returns a **value** which is a **response object** received by argument response2.
- ➢ The response object does not directly contain the actual response body but instead a representation of the entire HTTP response message. So, to extract the response body from the response object, use the **text** method, which returns

another **promise** that resolves with the result of parsing the response body to **string**.

➢ The promise again calls another then method which takes one argument **data** which contains the **string** format of the request body data. The method body calls **showStudent()** by sending data as an argument.

Display a single student

- Show a single student data which is received as a response from the Get request in a table in the frontend.

2.5. Implement the showStudent() as shown below.

```
function showStudent(data) {
    const student = JSON.parse(data)
    // Find a <table> element with id="myTable":
     var table = document.getElementById("myTable");

     // Create an empty <tr> element and add to the last position of the table:
     var row = table.insertRow(table.length);

     // Insert new cells (<td> elements) at the 1st and 2nd position of the
"new" <tr> element:
     var td=[]
     for(i=0; i<table.rows[0].cells.length; i++){
         td[i] = row.insertCell(i);
     }
     // Add student detail to the new cells:
     td[0].innerHTML = student.stdid;
     td[1].innerHTML = student.fname;
     td[2].innerHTML = student.lname;
     td[3].innerHTML = student.email;
     td[4].innerHTML = '<input type="button" onclick="deleteStudent(this)"
value="delete" id="button-1">';
     td[5].innerHTML = '<input type="button" onclick="updateStudent(this)"
value="edit" id="button-2">';

}
```

➢ The **parse()** of the **JSON** package converts string data which is in the JSON format to JavaScript object.

Using catch method to handle error

- If the response received from the create (POST) request is not successful, how can you handle the error?
    - Throw the error by invoking the **Error** constructor. Pass the response status text as a message.
    - To catch this error, use the **catch** method. catch method takes one parameter e, which is the status text. The catch method shows an **alert** message of the status text to the user.

2.6. Implement catch() as shown below.

```
    }).then(response1 => {
        var sid = data.stdid
        if (response1.ok) {
            fetch('/student/'+sid)
            .then(response2 => response2.text())
            .then(data => showStudent(data))
        } else {
            throw new Error(response1.statusText)
        }
    }).catch(e => {
        alert(e)
    })
}
```

- The **statusText** read-only property of the **Response** interface contains the **status message** corresponding to the HTTP status code.

  Ex., 200 OK - OK is the status text.

Form Validation

- At the moment it allows sending POST requests even when the input fields are **empty**. To avoid this, check the input values before calling the API.

2.7. Add the code is shown below in the addStudent(). Create the **sid** variable as global in addStudent() instead of defining it in the then method.

```
    var sid = data.stdid
    if (isNaN(sid)) {
        alert("Enter valid student ID")
        return
    } else if (data.email == "") {
        alert("Email cannot be empty")
        return
    } else if(data.fname == ""){
        alert("first name cannot be empty")
        return
    }
```

Reset Input Values

- Once the student details are added and shown in the table, **reset** the input field values to initial state (empty).

2.8. Create a new function **resetform** to do this task.

```
// set form fields to empty
function resetform() {
    document.getElementById("sid").value = "";
    document.getElementById("fname").value = "";
    document.getElementById("lname").value = "";
    document.getElementById("email").value = "";
}
```

2.9. Call this function from addStudent function to reset form values after the data is successfully stored.

➢ Here is the complete addStudent function code.

```
function addStudent(){
    var data = {
       stdid : parseInt(document.getElementById("sid").value),
       fname : document.getElementById("fname").value,
       lname : document.getElementById("lname").value,
```

```
        email : document.getElementById("email").value
    }
    var sid = data.stdid
    if (isNaN(sid)) {
        alert("Enter valid student ID")
        return
    } else if (data.email == "") {
        alert("Email cannot be empty")
        return
    } else if(data.fname == ""){
        alert("first name cannot be empty")
        return
    }

    fetch('/student', {
        method: "POST",
        body: JSON.stringify(data),
        headers: {"Content-type": "application/json; charset=UTF-8"}
    }).then(response1 => {
        if (response1.ok) {
            fetch('/student/'+sid)
            .then(response2 => response2.text())
            .then(data => showStudent(data))
        } else {
            throw new Error(response1.statusText)
        }
    }).catch(e => {
        alert(e)
    })

    resetform();
}
```

Display all students' details

- Show all students information in a table in the frontend.

window.onload Property

- The **window** object represents the **browser** window. The **onload** property processes load events after the element has finished loading. This is used with the window element to execute a script after the webpage has completely loaded. The function that is required to be executed is assigned as the handler function to this property.

2.10. Add the following code at the start of the student.js file.

```
window.onload = function () {
    fetch('/students')
        .then(response => response.text())
        .then(data => showStudents(data));
}
```

- ➢ Here, an anonymous function is assigned as value to the onload property. The function body uses the fetch method to send a GET request to the API URL ***/students.***
- ➢ Once the response is received, the first then method gets that data from the response body.
- ➢ The second then method takes the response data and passes it to **showStudents()** for displaying.

2.11. Implement the showStudents() as shown below.

```
function showStudents(data) {
    const students = JSON.parse(data)
    students.forEach(stud => {
        // Find a <table> element with id="myTable":
        var table = document.getElementById("myTable");
        // Create an empty <tr> element and add to the last position of the
table:
        var row = table.insertRow(table.length);
        // Insert new cells (<td> elements) in new <tr> element:
        var td=[]
        // Iterate the loop till the row 0 cell length in the table
        for(i=0; i<table.rows[0].cells.length; i++){
            td[i] = row.insertCell(i);
        }
        // Add student detail to the new cells:
```

```
        td[0].innerHTML = studstdid;
        td[1].innerHTML = stud.fname;
        td[2].innerHTML = stud.lname;
        td[3].innerHTML = stud.email;
        td[4].innerHTML = '<input type="button" onclick="deleteStudent(this)"
value="delete" id="button-1">';
        td[5].innerHTML = '<input type="button" onclick="updateStudent(this)"
value="edit" id="button-2">';
    })
}
```

Reducing duplicate codes

- Discard the duplicate codes for adding data to the table in showStudents and showStudent functions.

2.12. Create a new function helper **newRow** that adds a new row data to the table. The function takes one parameter which is a single student data.

```
function newRow(student) {
    // Find a <table> element with id="myTable":
    var table = document.getElementById("myTable");
    // Create an empty <tr> element and add it to the last position of the
table:
    var row = table.insertRow(table.length);
    // Insert new cells (<td> elements) in new <tr> element:
    var td=[]
    // Iterate the loop till the row 0 cell length in the table
    for(i=0; i<table.rows[0].cells.length; i++){
        td[i] = row.insertCell(i);
    }
    // Add student detail to the new cells:
    td[0].innerHTML = student.stdid;
    td[1].innerHTML = student.fname;
    td[2].innerHTML = student.lname;
    td[3].innerHTML = student.email;
    td[4].innerHTML = '<input type="button" onclick="deleteStudent(this)"
```

```
value="delete" id="button-1">';
     td[5].innerHTML = '<input type="button" onclick="updateStudent(this)"
value="edit" id="button-2">';

}
```

2.13. Reuse this function in both **showStudent** and **showStudents** functions. Update functions as shown below.

```
function showStudent(data) {
    const student = JSON.parse(data)
    newRow(student)
}

function showStudents(data) {
    const students = JSON.parse(data)
    students.forEach(stud => {
        newRow(stud)
    });
}
```

Edit Student Details

- Edit the student details by sending the Put API request.
- If you look at the earlier code that adds data to the table row, the **edit** onclick event of the button calls the **UpdateStudent()** and passes **this** object as an argument.

Q. What is **this** object in that statement?

- Here **this** object refers to the HTML **input** element.

2.14. Create a new function **updateStudent.** The function has one parameter, which is the **object** that called the function.

- This function simply retrieves the data from the selected row and displays it in the update form. Further, it updates the form button to call update() which actually sends the Put API request.

- ➢ In the function body firstly, get the row data of a particular student you want to update and store it in a variable **selectedRow**. You can declare the selectedRow as a global variable to use in the other functions.
- ➢ Next, fill in the form fields with the selected row data.
- ➢ Change button text from add to **Update**.
- ➢ On clicking the Update button, call another function update() by passing the student ID as an argument.

2.15. Implement the function as shown below.

```
var selectedRow = null;
function updateStudent(r) {
    selectedRow = r.parentElement.parentElement;
    // fill in the form fields with selected row data
    document.getElementById("sid").value = selectedRow.cells[0].innerHTML;
    document.getElementById("fname").value = selectedRow.cells[1].innerHTML;
    document.getElementById("lname").value = selectedRow.cells[2].innerHTML;
    document.getElementById("email").value = selectedRow.cells[3].innerHTML;

    var btn = document.getElementById("button-add");
    sid = selectedRow.cells[0].innerHTML;
    if (btn) {
        btn.innerHTML = "Update";
        btn.setAttribute("onclick", "update(sid)");
    }
}
```

2.16. Create another function **update** as shown below.

```
function update(sid) {


}
```

- This function reads the new form data submitted by the client and sends a Put request to the API.

2.17. Since the form data needs to be retrieved repeatedly, create a helper function **getFromData** so that you can reuse the function and reduce the code.

```
function getFormData() {
    var formData = {
        stdid : parseInt(document.getElementById("sid").value),
        fname : document.getElementById("fname").value,
        lname : document.getElementById("lname").value,
        email : document.getElementById("email").value
    }
    return formData
}
```

2.18. Implement update() as shown below.

```
function update(sid) {
    // data to be sent to the UPDATE request
   var newData = getFormData()
   fetch('/student/'+sid, {
       method: "PUT",
       body: JSON.stringify(newData),
       headers: {"Content-type": "application/json; charset=UTF-8"}
   }).then (res => {
        if (res.ok) {
            // fill in selected row with updated value
            selectedRow.cells[0].innerHTML = newData.stdid;
            selectedRow.cells[1].innerHTML = newData.fname;
            selectedRow.cells[2].innerHTML = newData.lname;
            selectedRow.cells[3].innerHTML = newData.email;
            // set to previous value
            var button = document.getElementById("button-add");
            button.innerHTML = "Add";
            button.setAttribute("onclick", "addStudent()");
            selectedRow = null;

            resetform();
        }else {
            alert("Server: Update request error.")
        }
   })
```

```
}
```

- ➢ update() uses the above function to get new data.
- ➢ Use the fetch method to send the update request to the server.
- ➢ If the response is a success, display the updated data in the table.
- ➢ Reset the form to initial state.

2.19. Update **addStudent** function by using **getFormData** function.

```
function addStudent(){
    var data = getFormData()
    fetch('/student', {
        method: "POST",
        body: JSON.stringify(data),
        headers: {"Content-type": "application/json; charset=UTF-8"}
    }).then(response1 => {
        if (response1.ok) {
            fetch('/student/'+sid)
            .then(response2 => response2.text())
            .then(data => showStudent(data))
        } else {
            throw new Error(response1.statusText)
        }
    }).catch(e => {
        alert(e)
    })

    resetform();
}
```

Delete a single student data

- Similar to the edit button, the delete button calls deleteStudent() by passing this object.

2.20. Create a function **deleteStudent.** The function has one parameter which is the object that invoked the function.

- ➢ Before sending the delete request, **alert** the user to confirm the action.
- ➢ Get the row data of the student you want to delete.

- ➢ Send the delete request using the fetch method.
- ➢ If the delete request is successful, delete the particular row from the table.
- ➢ Reset selectedRow value to null.
- ➢ Here is the code.

```
function deleteStudent(r){
    // this(input) -> td -> tr
    if (confirm('Are you sure you want to DELETE this?')){
        selectedRow = r.parentElement.parentElement;
        sid = selectedRow.cells[0].innerHTML;

        fetch('/student/'+sid, {
            method: "DELETE",
            headers: {"Content-type": "application/json; charset=UTF-8"}
        });
        var rowIndex = selectedRow.rowIndex; // index starts from 0
        if (rowIndex>0) { //th is row 0
            document.getElementById("myTable").deleteRow(rowIndex);
        }
        selectedRow = null;
    }
}
```

- ➢ Test your application and verify the result.

Sign Up and Login

Sign up

2.21. Open signup.js file and define a new function **signUp**.

Q. How to implement the sign up functionality?

- ➢ Read the user data from the sign up form.
- ➢ Check if both the password matches before sending the signup request.
- ➢ Send the sign up request to the server.
- ➢ If the response status is a success, show the login page.

2.22. Implement the function as shown below.

```
function signUp(){
    // data to be sent to the POST request
    var _data = {
        firstname : document.getElementById("fname").value,
        lastname : document.getElementById("lname").value,
        email : document.getElementById("email").value,
        password : document.getElementById("pw1").value,
        pw : document.getElementById("pw2").value
    }
    if (_data.password !== _data.pw) {
        alert("PASSWORD doesn't match!")
        return
    }
    fetch('/signup', {
        method: "POST",
        body: JSON.stringify(_data),
        headers: {"Content-type": "application/json; charset=UTF-8"}
    })
    .then(response => {
        if (response.status == 201) {
            // console.log("logged in")
            window.open("login.html", "_self")
        }
    });
}
```

2.23. In the signup.html file add **onclick** property to the submit button and import the signup.js file.

Login

2.24. Open login.js file, define function **login.** Implement the login function as shown below.

```
function login() {
    var _data = {
        email : document.getElementById("email").value,
```

```
        password : document.getElementById("pw").value
    }

    fetch('/login', {
        method: "POST",
        body: JSON.stringify(_data),
        headers: {"Content-type": "application/json; charset=UTF-8"}
    })
    .then(response => {
        if (response.ok) {
            window.open("student.html", "_self")
        } else {
            throw new Error(response.statusText)
        }
    }).catch(e => {
        if (e == "Error: Unauthorized") {
            alert(e+ ". Credentials does not match!")
            return
        }
    });
}
```

- **_self** tells the browser to open the web page in the same tab.

2.25. In the login.html file add **onclick** property to the submit button and import the login.js file.

Handling indexes

- An index is a list of files and folders contained in a directory. When a directory does not contain an **index.html** file, these servers take the responsibility of returning HTML with a **list** of these files and folders.

- This is exactly what Go file server is doing. However, you do not want a user to see the content of the directory. In that case, you should return HTML with a directory index. You can do that by placing **index.html** in every directory that can return an index.

2.26. **Rename** the **login.html** file to **index.html** contained in the root directory served by your HTTP server.

➢ Start the server again to update the changes.

#Reminder push your source code to the GitLab repository.

3. Sign up and Login API

Creating Admin Table

- Before creating the APIs, create an admin table in the database to store admin details.

3.1. Create a new table **admin** in my_db database. The SQL query is given below.

```
CREATE TABLE admin (
    FirstName varchar(45) NOT NULL,
    LastName varchar(45) DEFAULT NULL,
    Email varchar(45) NOT NULL,
    Password varchar(45) NOT NULL,
    PRIMARY KEY (Email)
)
```

Sign up

3.2. In the routes.go define route **/signup** in the mux router and map to handler function **Signup**.
3.3. Create a new file **admin.go** (controller/admin.go) in the controller package.
3.4. Define a new handler function **Signup**.
3.5. Create a new file **admin.go** ((myapp/model/admin.go) in model package. Inside this file create a new struct type **Admin** to represent an admin object.

```go
package model

type Admin struct {
    FirstName string
    LastName  string
    Email     string
    Password  string
}
```

Q. How to implement the SignUp handler function?

- ➢ Get the sign up user data from the request body.
- ➢ Decode the JSON object and store it in **Admin** variable type.
- ➢ Send the data to model by calling the **Create** function.
- ➢ Send HTTP response back to the user.

3.6. Implement the function as shown below.

```
func Signup(w http.ResponseWriter, r *http.Request) {
    var admin model.Admin
    decoder := json.NewDecoder(r.Body)
    if err := decoder.Decode(&admin); err != nil {
        httpResp.RespondWithError(w, http.StatusBadRequest, "invalid json body")
        return
    }
    defer r.Body.Close()
    saveErr := admin.Create()
    if saveErr != nil {
        httpResp.RespondWithError(w, http.StatusBadRequest, saveErr.Error())
        return
    }
    // no error
    httpResp.RespondWithJSON(w, http.StatusCreated, map[string]string{"status":
"admin added"})
}
```

Implementing Model (model/admin.go)

3.7. Define a receiver function **Create** which receives an Admin reference type. The function returns an error interface type.

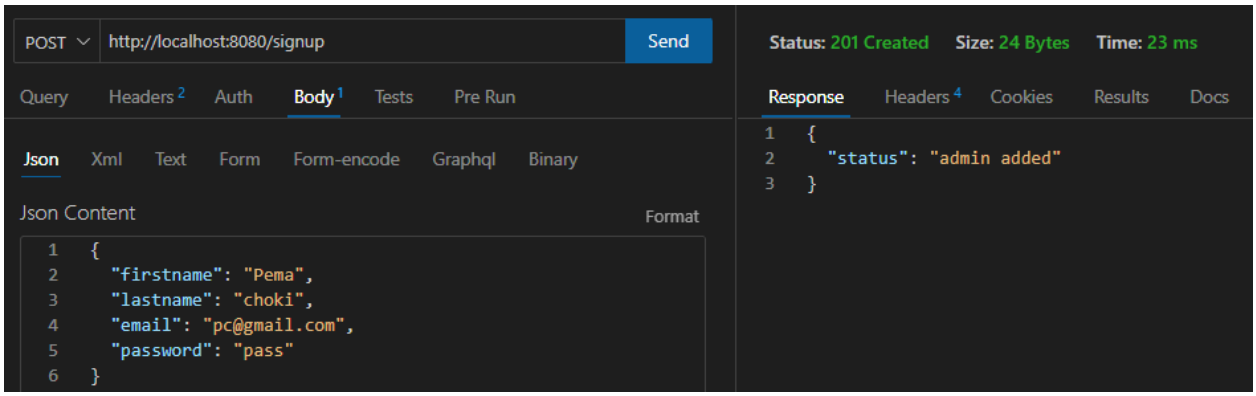
- ➢ Define SQL query to **insert** admin data to database.
- ➢ Send the request to the database using QueryRow().
- ➢ Here's the code.

```
const queryInsertAdmin = "INSERT INTO admin(firstname, lastname, email,
password) VALUES($1, $2, $3, $4) RETURNING email;"

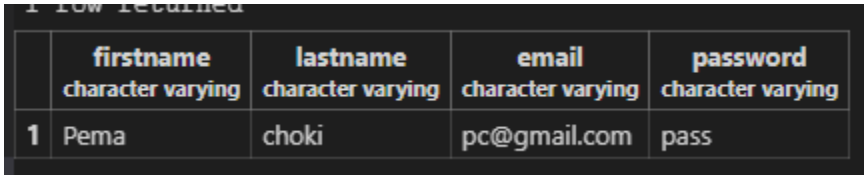
func (adm *Admin) Create() error {
```

```
    row := postgres.Db.QueryRow(queryInsertAdmin, adm.FirstName,
adm.LastName, adm.Email, adm.Password)
    err := row.Scan(&adm.Email)
    return err
}
```

3.8. Test the API. You can also test from the sign up user interface.

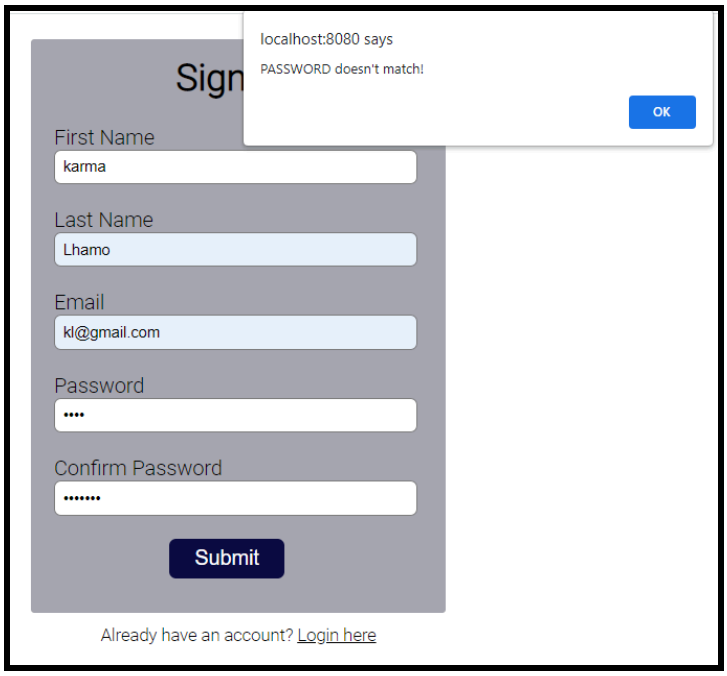


admin table in the database

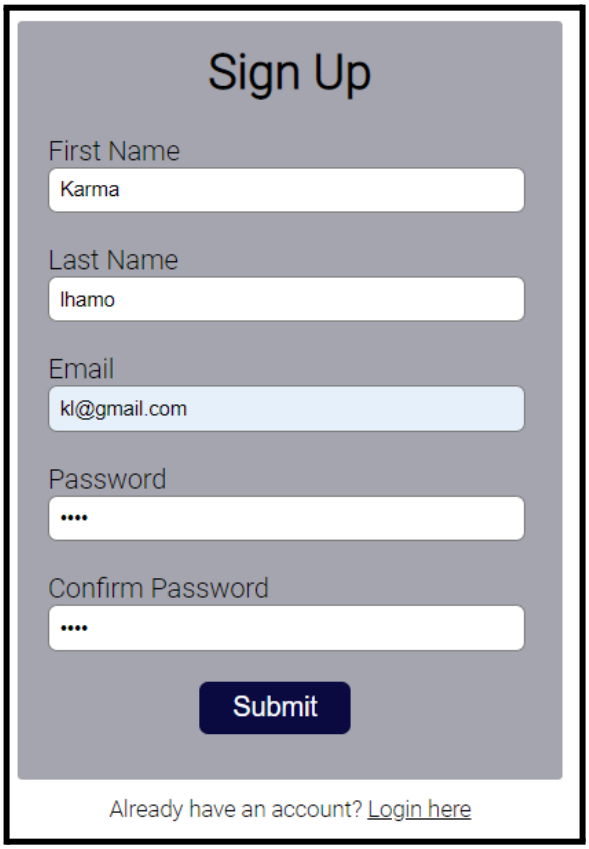


	firstname character varying	lastname character varying	email character varying	password character varying
1	Pema	choki	pc@gmail.com	pass

➢ If the two passwords do not match, it gives an alert message.



➢ Using Sign up UI



➢ Data stored in the admin table in the database.

2 rows returned

	firstname character varying	lastname character varying	email character varying	password character varying
1	Pema	choki	pc@gmail.com	pass
2	karma	lhamo	kl@gmail.com	pass

Login

3.9. In the routes.go define route **/login** in the mux router and map to handler function **Login**.

3.10. In the admin.go file define the handler function **Login** (controller/admin.go). Implement handler function as shown below.

```
func Login(w http.ResponseWriter, r *http.Request) {
    var admin model.Admin
    err := json.NewDecoder(r.Body).Decode(&admin)
    if err != nil {
        httpResp.RespondWithError(w, http.StatusBadRequest, "invalid json
body")
        return
    }
    defer r.Body.Close()

    getErr := admin.Get()
    if getErr != nil {
        httpResp.RespondWithError(w, http.StatusUnauthorized, getErr.Error())
        return
    }
    httpResp.RespondWithJSON(w, http.StatusOK, map[string]string{"message":
"success"})
}
```

Implementing Model (model/admin.go)

3.11. Define a receiver function **Get** which receives an Admin reference type. The function returns an error interface type.

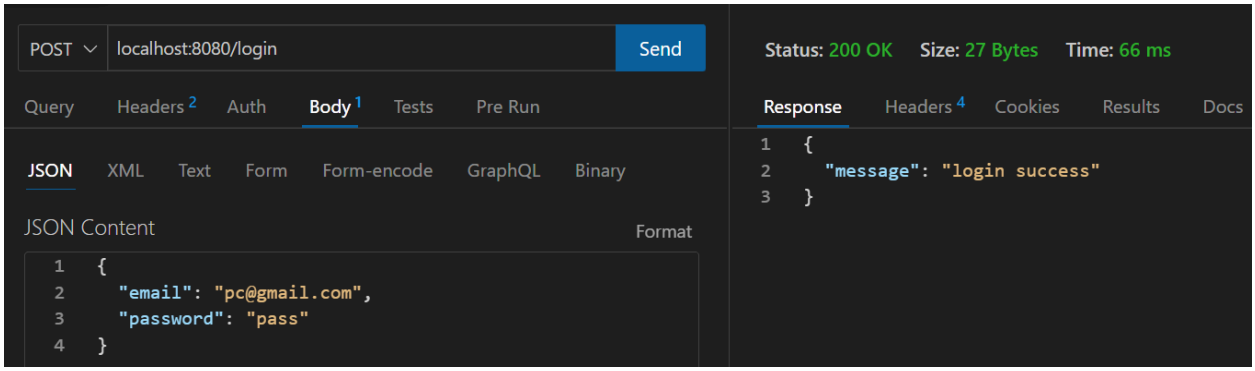
➢ Define SQL query to **get** admin data from database.

- Send the request to the database using QueryRow().
- Here's the code.

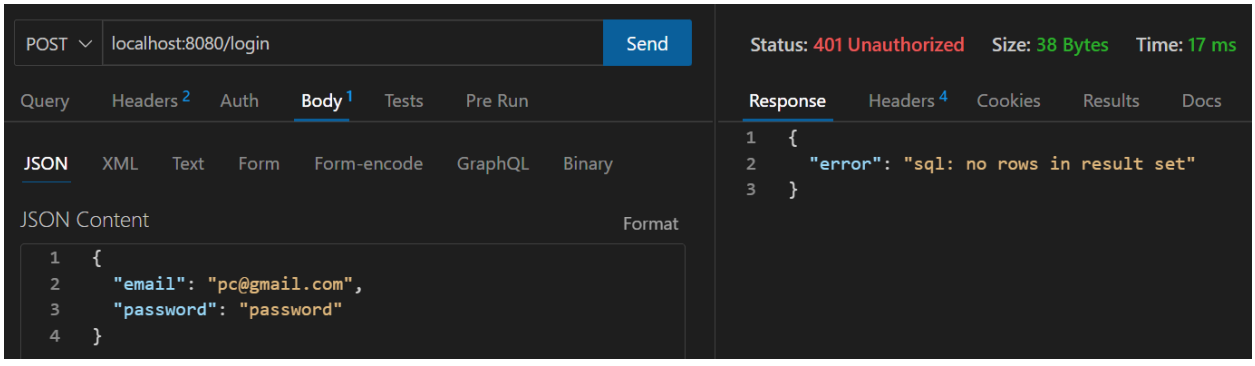
```
const queryGetAdmin    = "SELECT email, password FROM admin WHERE email=$1 and
password=$2;"

func (adm *Admin) Get() error {
    return postgres.Db.QueryRow(queryGetAdmin, adm.Email,
adm.Password).Scan(&adm.Email, &adm.Password)
}
```

3.12. Test the login API.



- When credentials do not match, you get http response status code 401 unauthorized.



Well done! 🙌🙌 A simple login and sign up functionality is successfully implemented.

4. Session Management

- Currently, the login functionality does not make your application fully secure. It does **verify** the user credentials and give the access accordingly however, if you access other web pages (say course.html) without logging in, you can view the page and perform CRUD actions.

  Q. How can you authenticate the user and maintain the session across different pages until the logout? How to secure your web API ?

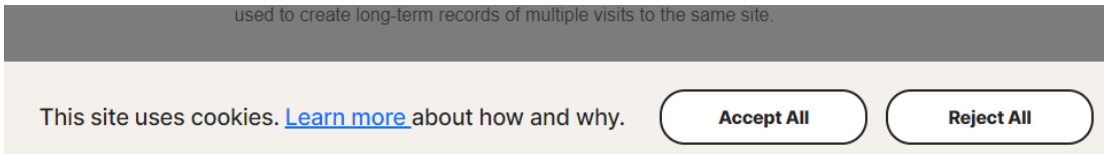
- "**HTTP**" is a **"stateless"** protocol, which means every HTTP request made to a **server** is like a fresh call, with no memory or **"state"** of the previous requests.
- In applications you generally need to maintain **"sessions"**, so that once a user logs in, they can continue to access different parts of the application without having to log in (i.e. authenticate) again and again for each subsequent "HTTP" request made to the server.

- There are primarily two different approaches to session management,
  - **Session or Cookies based approach**
  - JWT (JSON Web Tokens) based approach

Cookie

Q. What is a cookie on the web?

- You must have encountered a cookie like shown below.



- **Cookies** are small files of **information** that a **web server creates** and **sends to a web browser**.
  (OR) Computer **cookies** are small files, often including unique identifiers that web servers send to browsers.
- Web browsers **store** the cookies they receive for a **predetermined period** of time, or for the length of a user's session on a website. Browsers **attach** the relevant cookies to any **future requests** made to that web server. This way cookies help **inform** websites about the user, their preferences, and thus, enables the websites to **personalize** the user experience.
- For example, ecommerce websites use cookies to know what merchandise users have placed in their shopping carts and accordingly similar items are shown frequently.

- On the other hand, some cookies are used for **security** purposes, such as authentication cookies.

  Q. Where are cookies stored?

- **Web browsers** store cookies in a designated file on users' devices.

  For example, Google Chrome stores all cookies in a file labeled "**Cookies.**" Chrome users can view the cookies stored by the browser by opening **developer tools,** clicking the "Application" tab, and clicking on "Cookies" in the left side menu.

- The cookies that are used on the Internet are also called "**HTTP cookies**." Like much of the web, cookies are sent using the HTTP protocol.

  Q. What are Cookies used for?

1. **Session Management**

- Cookies help to associate website **activity** of a specific user. A session cookie contains a unique string (a combination of letters and numbers) that matches a user session with relevant data and content for that user.

  Ex., Suppose Alice has an account on a shopping website. She logs into her account from the website's homepage. When she logs in, the website's server generates a session cookie and sends the cookie to Alice's browser. This cookie tells the website to load Alice's account content, so that the homepage now reads, "Welcome, Alice."
  Alice then clicks to a product page displaying a pair of jeans. When Alice's web browser sends an HTTP request to the website for the jeans product page, it includes Alice's session cookie with the request. Because the website has this cookie, it recognizes the user as Alice, and she does not have to log in again when the new page loads.

2. **Personalization**

- Cookies help a website "**remember**" user actions or user preferences, enabling the website to **customize** the user's experience.

- If Alice logs out of the shopping website, her username can be stored in a cookie and sent to her web browser. Next time she loads that website, the web browser sends this cookie to the web server, which then prompts Alice to log in with the username she used last time.

3. **Tracking**

- Some cookies **record** what websites users visit. This information is sent to the server that originated the cookie the next time the browser has to load content from that server. With third-party tracking cookies, this process takes place anytime the browser loads a website that uses that tracking service.

- If Alice has previously visited a website that sent her browser a tracking cookie, this cookie may record that Alice is now viewing a product page for jeans. The next time Alice loads a website that uses this tracking service, she may see ads for jeans.

- However, advertising is not the only use for tracking cookies. Many analytics services also use tracking cookies to anonymously record user activity.

Types of Cookie

1. **Session Cookie**

- A session cookie helps a website track a **user's session**. Session cookies are **deleted** after a user's session ends - once they log out of their account on a website or exit the website. Session cookies have no expiration date, which signifies to the browser that they should be deleted once the session is over.

2. **Persistent Cookie**

- Persistent cookies are those that remain in the browser for an **extended** length of time. They will only be erased when the cookies expire or the users clear them from the browser after being installed.

3. **First-party Cookie**

- Created by the website that the user visits. These are generally safer, as long as it is a reputable website or ones that have not been compromised.

4. **Third-party Cookie**

- They are created by websites that are **different** from the web pages users are currently surfing, usually because they're linked to **ads** on that page.
  Visiting a site with 10 ads may generate 10 cookies, even if users never click on those ads.
- It is easier for parties you can't identify to watch what sites you are visiting, what files you are downloading, and what images you are clicking on. Usually for tracking and personalization purposes.

Q. How to disable third-party cookies for online privacy?

The process depends on the type of browser you use.

For Chrome, click the three dots in the upper right-hand corner of the browser. Next, click "Settings." In this menu, click "Privacy and security." In this menu, you can choose

the radio button next to “Block third-party cookies" to stop third-party cookies from tracking your online activity.

5. Zombie Cookie
6. Flash Cookie
7. Secure Cookie
   When setting the **Secure** field to **true**, the cookie will only be sent over **HTTPS**. This means that you need to make sure that your server is configured to handle HTTPS requests properly, and that the client is accessing your site over HTTPS.

8. Super Cookie

9. **HTTPOnly Cookie**
   HttpOnly: True
   To prevent it from being accessed by client-side JavaScript, which can help mitigate against cross-site scripting (XSS) attacks.

Session Management Using Cookie

- In your application, you will create a cookie for session management across all the pages. Session is created when the user login.

- To create a cookie in Go, **http** package has a struct type called **Cookie** which represents the web cookie. It is defined as shown below.

```
type Cookie struct {
    Name  string
    Value string

    Path       string
    Domain     string
    Expires    time.Time
    RawExpires string

    // MaxAge=0 means no 'Max-Age' attribute specified.
    // MaxAge<0 means delete cookie now, equivalently 'Max-Age: 0'
    // MaxAge>0 means Max-Age attribute present and given in
seconds
    MaxAge   int
    Secure   bool
    HttpOnly bool
    SameSite SameSite
    Raw      string
    Unparsed []string
```

```
}
```

- To create cookies, you can create an **instance** of **Cookie** type. Set necessary fields such as **name, value, expiration time**, and **secure**. Store it in a variable cookie which is of type Cookie.

4.1. Add the following code in the **Login** handler function right after the user authentication is successful.
Here the cookie value is simply a dummy string. Cookie lifetime is set to 30 minutes.

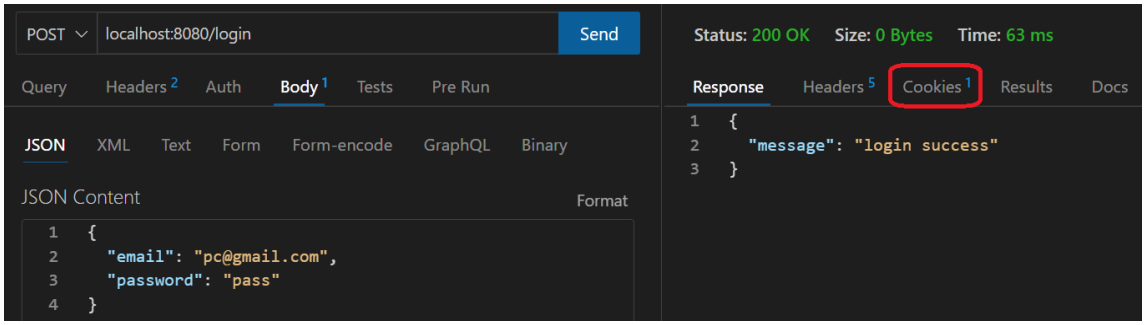
```
        // fmt.Println(getErr.Error())
        httpResp.RespondWithError(w, http.StatusUnauthorized, ge
        return
    }

    // set cookie
    cookie := http.Cookie{
        Name:    "my-cookie",
        Value:   "my-value",
        Expires: time.Now().Add(30 * time.Minute),
        Secure:  true,
    }
    http.SetCookie(w, &cookie)

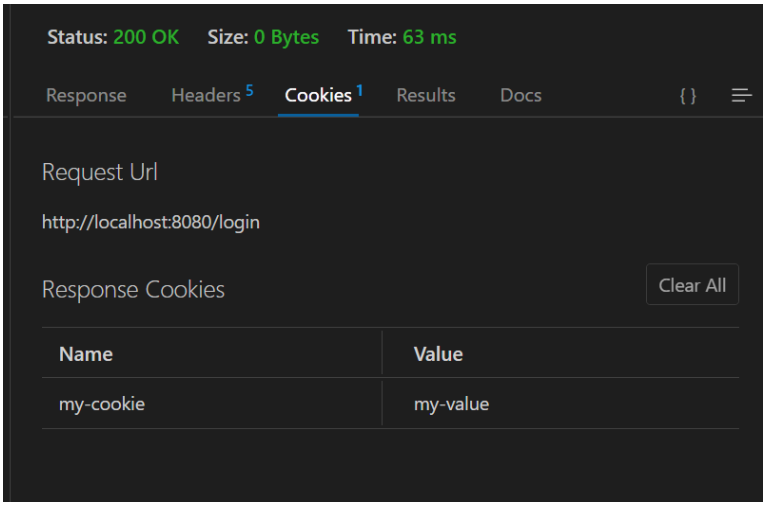
    httpResp.RespondWithJSON(w, http.StatusOK, map[string]string
}
```

- **SetCookie()** in the http package is to send a cookie in HTTP response.

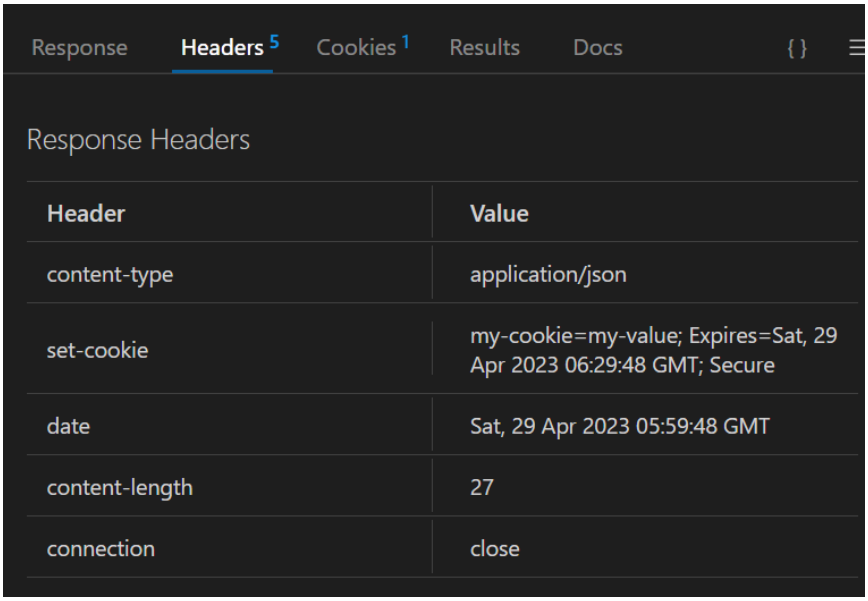
4.2. Test the login API and verify if the cookie is set during login.



➢ Click on the **Cookies** menu to see the cookie information like name and value.



➢ Click on the **Headers** menu to see detailed cookie information.



➢ You can also check the cookie by looking into your browser's cookie storage. For that use a web browser to send a login request.

4.3. Now **restrict** the user to **access** the web pages only if the cookie is set. For this create a new JS file **auth.js** and write the following code.

```
if (document.cookie == "") {
    alert("User not logged in!!")
    window.open("index.html", "_self")
} else {
```

```
    console.log("cookie set")
}
```

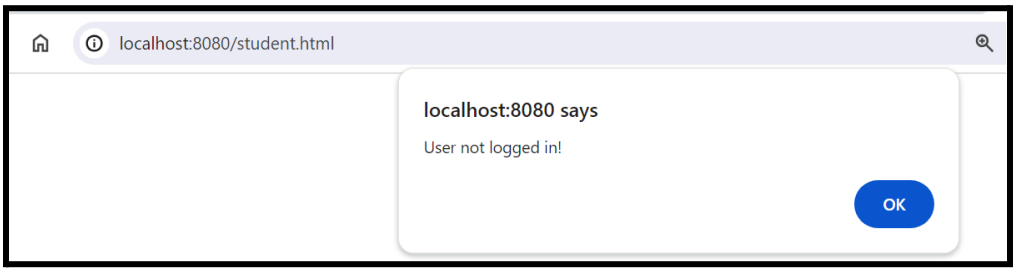
- JavaScript can create, read, and delete cookies with the **document.cookie** property.
- document.cookie returns a **string** that contains all of the cookies associated with the current page. A single page can contain more than one cookie. Each cookie is represented as a **key-value** pair, separated by a semicolon and a space.

  For example: cookie1=value; cookie2=value; cookie3=value;

➢ In the above code, document.Cookie property reads the cookie from the browser and verifies if the cookie value is an empty string. If the cookie value is empty (cookie not set), **alert** the user and redirect the user to **index.html** page to let the user login first.

4.4. **Import** the **auth.js** file in the three html pages to check the cookie before loading the page.

➢ Now go to the browser and check if you are able to access any web pages before login. The user must see an alert message and be redirected to the login page.



Logout Handler

- When the user logs out, make sure the session duration expires. For that you must **delete** the cookie.
- One of the ways to **destroy** the cookie is to set the cookie **expiration time** to the **current time**.

4.5. In the routes.go define a new route **"/logout"** and map the handler function **Logout.**

```
router.HandleFunc("/login", controller.Login).Methods("P
router.HandleFunc("/logout", controller.Logout)
```

4.6. In the admin.go file define a handler function **Logout** and add the following code.

```
func Logout(w http.ResponseWriter, r *http.Request) {
    http.SetCookie(w, &http.Cookie{
        Name:    "my-cookie",
        Expires: time.Now(),
    })

    httpResp.RespondWithJSON(w, http.StatusOK, map[string]string{"message":
"cookie deleted"})
}
```

4.7. In the login.js file create a new function **logout**. Add the code below in the function body.

```
function logout() {
    fetch('/logout')
    .then(response => {
        if (response.ok) {
            window.open("index.html", "_self")
        } else {
            throw new Error(response.statusText)
        }
    }).catch (e => {
        alert(e)
    })
}
```

4.8. In the html files student, course, and enroll add the **onclick** property to the **Logout** list item and call the function logout. Also import login.js file in these pages.

4.9. Test the API in the browser and check if the cookie gets deleted. Restart the server first.

Securing Web API

- Secure each API by verifying the cookie at the server side before processing the request.

4.10. To verify the cookie, define a new function **VerifyCookie** in the **admin.go** file.

The complete function body is given below.

```
func VerifyCookie(w http.ResponseWriter, r *http.Request) bool {
    // Retrieve the "my-cookie" cookie from the request
    cookie, err := r.Cookie("my-cookie")
    if err != nil {
        if err == http.ErrNoCookie {
            // No cookie found, redirect to login page or return an error
            httpResp.RespondWithError(w, http.StatusSeeOther, "cookie not
found")
            return false
        }
        // Some other error occurred
        httpResp.RespondWithError(w, http.StatusInternalServerError,
"internal server error")
        return false
    }

    // Verify the cookie value
    if cookie.Value != "my-value" {
        // Invalid cookie value, redirect to login page or return an error
        httpResp.RespondWithError(w, http.StatusUnauthorized, "cookie does
not match")
        return false
    }
    return true
}
```

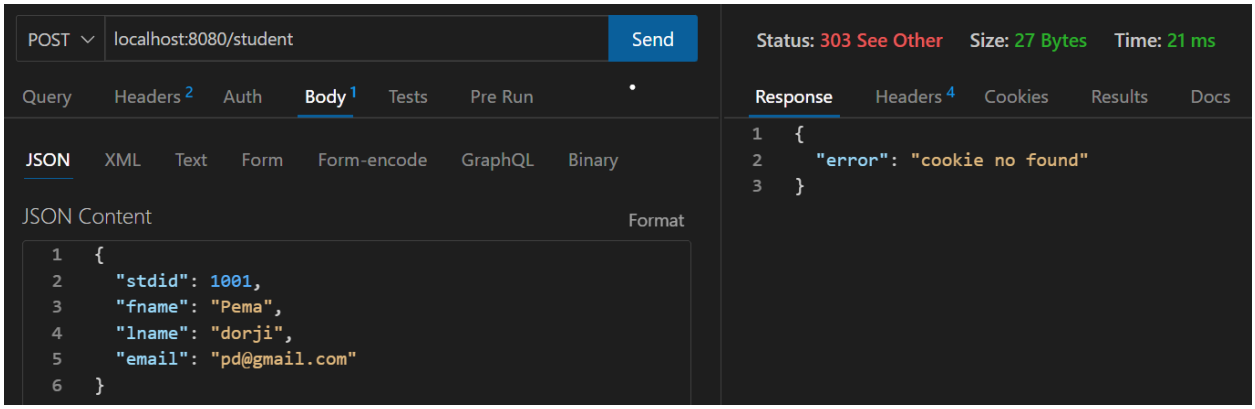
- The function has two parameters, response writer interface and a Request pointer. The function returns a **bool** true if the cookie is valid and false otherwise.
- The function sends HTTP response status see other and internal server error if the cookie is not set or cookie value does not match.

4.11. Now verify the cookie by calling this function from the handler functions before processing the request. Add the following code at the start of each handler function of student, course, and enroll.

```
func AddStudent(w http.ResponseWriter, r *http.Request) {
    // validate cookie
    if !VerifyCookie(w, r) {
        return
    }

    var stud model.Student
    // extracting data from request body
```

4.12. Test the code by sending the POST request without logging in.



4.13. In the student.js file, update the catch method by checking the response status code and handle the error accordingly.

```
.then(response => {
        if (response.ok) {
            fetch('/student/'+sid)
            .then(response => response.text())
            .then(data => showStudent(data))
        } else {
            throw new Error(response.status) // new keyword constructs a
this obj and assign {name:"Error", message:""}
        }
    }).catch(e => {
        if (e.message == 303) {
            alert("User not logged in.")
            window.open("index.html", "_self")
        } else if (e.message == 500) {
            alert("Server error!")
        }
    });
```

- The **status** read-only property of the **Response** interface contains the HTTP **status codes** of the response.
- The **message** property sets or returns an error message.

5. Enroll APIs

5.1. Within the **model** package create a new file **enroll.go** (model/enroll.go).
5.2. Inside the file, create a user defined type **Enroll** to represent the Enroll object.

Here is the code, enroll.go.

```
package model

type Enroll struct {
	StdId         int64  `json:"stdid"`
	CourseID      string `json:"cid"`
	Date_Enrolled string `json:"date"`
}
```

1. Add Enrollment

- Enroll a student to a particular course.

5.3. In the routes.go file, define a new route **"/enroll"** to enroll a student and map it to a handler function **Enroll**. The API method is POST.

Date package

➢ As you know, the enroll type has a field Date_enrolled to store the date of enrollment. To get the date, you are going to create your own **date** package that gives us the **current date**.

5.4. Within the **utils** directory, create a new folder **date** (new package). Within the date package, create a new file **date.go** and define the function **GetDate** as shown below.

```
package date
```

```
import "time"

const (
    apiDateLayout = "2006-01-02T15:04:05Z"
)

func GetDate() string {
    date := time.Now().UTC()          // get current date and time
    return date.Format(apiDateLayout) // convert Time type to string
}
```

5.5. Now moving to the controller, create a new file **enroll.go** (controller/enroll.go) in the controller package and define the handler function **Enroll** as shown below.

```
func Enroll(w http.ResponseWriter, r *http.Request) {
    var e model.Enroll
    decoder := json.NewDecoder(r.Body)
    if err := decoder.Decode(&e); err != nil {
        httpResp.RespondWithError(w, http.StatusBadRequest, "invalid json
body")
        return
    }

    e.Date_Enrolled = date.GetDate()
    defer r.Body.Close()

    saveErr := e.EnrollStud()
    if saveErr != nil {
        if strings.Contains(saveErr.Error(), "duplicate key") {
            httpResp.RespondWithError(w, http.StatusForbidden, "Duplicate
keys")
            return
        } else {
            httpResp.RespondWithError(w, http.StatusInternalServerError,
saveErr.Error())
            return
        }
```

```
    }
    // no error
    httpResp.RespondWithJSON(w, http.StatusCreated, map[string]string{"status":
"enrolled"})
}
```

5.6. In the model package (enroll.go) define a new receiver function **EnrollStud.**

Here is the code.

```
const (
    queryEnrollStd = "INSERT INTO enroll(std_id, course_id,
date_enrolled) VALUES($1, $2, $3) RETURNING std_id;"
)

func (e *Enroll) EnrollStud() error {
    row := postgres.Db.QueryRow(queryEnrollStd, e.StdId, e.CourseID,
e.Date_Enrolled)
    err := row.Scan(&e.StdId)
    return err
}
```

2. Get Enrollment

- Get a particular student enrolled in a particular course.

5.7. Define a new route **"/enroll/{sid}/{cid}"** to enroll a student to a course and map it to a handler function **GetEnroll**. The API method is GET.

5.8. In the enroll controller package, define the function **GetEnroll.**

Here is the code.

```
func GetEnroll(w http.ResponseWriter, r *http.Request) {
    // get url parameters
    sid := mux.Vars(r)["sid"]
    cid := mux.Vars(r)["cid"]
    // get string sid to int type
    stdid, _ := strconv.ParseInt(sid, 10, 64)

    e := model.Enroll{StdId: stdid, CourseID: cid}
    getErr := e.Get()
    if getErr != nil {
        switch getErr {
        case sql.ErrNoRows:
            httpResp.RespondWithError(w, http.StatusNotFound, "No such
enrollments")
        default:
            httpResp.RespondWithError(w, http.StatusInternalServerError,
getErr.Error())
        }
        return
    }
    httpResp.RespondWithJSON(w, http.StatusOK, e)
}
```

5.9. In the enroll model package, define the receiver function **Get.** The code is shown below.

```
queryGetEnroll          = "SELECT std_id, course_id, date_enrolled FROM enroll WHERE std_id=$1 and course_id=$2;"
```

```
func (e *Enroll) Get() error {
    return postgres.Db.QueryRow(queryGetEnroll,
        e.StdId, e.CourseID).Scan(&e.StdId, &e.CourseID, &e.Date_Enrolled)

}
```

3. Get All Enrollment

- Get a list of students enrolled.

5.10. Define a new route **"/enrolls"** to enroll a student to a course and map it to a handler function **GetEnrolls**. The API method is GET.

5.11. In the enroll controller package, define the function **GetEnrolls.**

Here is the code.

```
func GetEnrolls(w http.ResponseWriter, r *http.Request) {
    enrolls, getErr := model.GetAllEnrolls()
    if getErr != nil {
        httpResp.RespondWithError(w, http.StatusBadRequest, getErr.Error())
        return
    }
    httpResp.RespondWithJSON(w, http.StatusOK, enrolls)
}
```

5.12. In the enroll model package, define the function **GetAllEnrolls.** The code is shown below.

```
func GetAllEnrolls() ([]Enroll, error) {
    rows, getErr := postgres.Db.Query("SELECT std_id, course_id, date_enrolled
from enroll;")
    if getErr != nil {
        return nil, getErr
    }
    // create a slice of type Course
    enrolls := []Enroll{}

    for rows.Next() {
        var e Enroll
        dbErr := rows.Scan(&e.StdId, &e.CourseID, &e.Date_Enrolled)
        if dbErr != nil {
            return nil, dbErr
        }
        enrolls = append(enrolls, e)
    }
    rows.Close()
    return enrolls, nil
}
```

4. Delete Enrollment

- Delete a particular student enrolled in a particular course.

5.13. Define a new route **"/enroll/{sid}/{cid}"** to enroll a student to a course and map it to a handler function **DeleteEnroll**. The API method is **DELETE**.

5.14. In the enroll controller package, define the function **DeleteEnroll.**

Here is the code.

```
func DeleteEnroll(w http.ResponseWriter, r *http.Request) {
    sid := mux.Vars(r)["sid"]
    cid := mux.Vars(r)["cid"]
    stdid, _ := strconv.ParseInt(sid, 10, 64)

    e := model.Enroll{StdId: stdid, CourseID: cid}

    if err := e.Delete(); err != nil {
        httpResp.RespondWithError(w, http.StatusBadRequest, err.Error())
        return
    }
    httpResp.RespondWithJSON(w, http.StatusOK, map[string]string{"status":
"deleted"})
}
```

5.15. In the enroll model package, define the receiver function **Delete.** The code is shown below.

```
queryDeleteEnroll = "DELETE FROM enroll WHERE std_id=$1 and course_id=$2
RETURNING std_id;"
```

```
func (e *Enroll) Delete() error {
    row := postgres.Db.QueryRow(queryDeleteEnroll, e.StdId, e.CourseID)
    err := row.Scan(&e.StdId)
    return err
}
```

You have completed all APIs required for enrollment.

#Reminder push your source code to the GitLab repository.

6. Enroll View

6.1. Open **enroll.js** file and define a new function **getStudents**.

➢ getStudents function creates a drop down list for the Student ID field in the enroll form. The list contains student IDs of all the students registered. It takes one parameter **data** of type string, which is the data in the response body.

Here is the code.

```
function getStudents(data) {
    const students = [] // to store all student ID
    const allStudents = JSON.parse(data)
    // add student id to students list
    allStudents.forEach(stud => {
        students.push(stud.stdid)
    });
    // get the sid input field
    var select = document.getElementById("sid")
    // iterate over the students and create a new option for each ID
    for (var i = 0; i < students.length; i++) {
        var sid = students[i];
        var option = document.createElement("option")
        option.textContent= sid;
        option.value = sid;
        select.appendChild(option);
    }
}
```

6.2. Define another function **getCourses** to create a drop down list for Course ID field in the enroll form. It takes one parameter **data** of type string, which is the response body.

Here is the code.

```
function getCourses(data) {
    const courses = []
    const allCourses = JSON.parse(data)
    allCourses.forEach(course => {
        courses.push(course.cid)
    });

    var option = "";
    for (var i = 0; i < courses.length; i++) {
        option += '<option value="'+ courses[i]+'">'+courses[i]+"</option>"
    }
    document.getElementById("cid").innerHTML = option;
}
```

6.3. Use **window.onload** property to get all students and courses and display student IDs and course IDs in the list as soon as you load the page.

```
window.onload = function () {
    // request to get all students
    fetch('/students')
        .then(response => response.text())
        .then(data => getStudents(data));
    // get all courses
    fetch('/courses')
        .then(response => response.text())
        .then(data => getCourses(data));
}
```

1. Add Enrollment

➢ Send the enrollment request to the server.

6.4. Open **enroll.html** and add **onclick** property to the enroll button.
6.5. Import **enroll.js** script file in enroll page.
6.6. In the enroll.js define a new function **addEnroll** to send enrollment requests to the server.

Here is the code.

```
function addEnroll(){
    var _data = {
        stdid : parseInt(document.getElementById("sid").value),
        cid : document.getElementById("cid").value,
    }
    var sid = _data.stdid;
    var cid = _data.cid;

    if (isNaN(sid) || cid == "") {
        alert("Select valid data")
        return
    }
    fetch('/enroll', {
        method: "POST",
        body: JSON.stringify(_data),
        headers: {"Content-type": "application/json; charset=UTF-8"}
    });
}
```

2. Show Enrollment

➢ If the enrollment POST request is successful, show the particular enrollment details in the table.

6.7. For this create a new function **getEnrolled** in the enroll.js file. The function receives one argument which is the HTTP response body.

Here's the code.

```
function getEnrolled(data) {
    const enrolled = JSON.parse(data)
    showTable(enrolled)

}
```

6.8. Create a helper function **showTable** to add a new row to display a single student data.

```
function showTable(enrolled) {
    // Find a <table> element with id="myTable":
    var table = document.getElementById("myTable");
    // Create an empty <tr> element and add it to the last position of
the table:
    var row = table.insertRow(table.length);
    // Insert new cells (<td> elements) at the 1st and 2nd position of
the "new" <tr> element:
    var td=[]
    for(i=0; i<table.rows[0].cells.length; i++){
        td[i] = row.insertCell(i);
    }
    td[0].innerHTML = enrolled.stdid;
    td[1].innerHTML = enrolled.cid;
    td[2].innerHTML = enrolled.date.split("T")[0]; // show only date,
ignore time
    td[3].innerHTML = '<input type="button" onclick="deleteEnroll(this)"
value="Delete" id="button-1">';
}
```

6.9. Add a **then** method to the promise returned by **fetch** API call to "/enroll". The then method takes one parameter which is the response object. If the response status is OK, it uses another fetch method to send API requests to get particular enrollment details.

6.10. If the response status is not a success, throw the error. Then add a **catch** method to handle the error.

6.11. Reset the form values after displaying the data.

Here is the code.

```
    .then(response => {
        if (response.ok) {
            fetch('/enroll/'+sid+'/'+cid)
            .then(response => response.text())
            .then(data => getEnrolled(data))
        } else {
            throw new Error(response.statusText)
```

```
        }
    }).catch(e => {
        if (e == "Error: Forbidden") {
            alert(e+". Duplicate entry!")
        }
    });
    resetFields();
```

6.12. Define resetFields() as shown below.

```
function resetFields() {
    document.getElementById("sid").value = "";
    document.getElementById("cid").value = "";
}
```

➢ Test the functionality by launching the application in the browser.

3. Get All Enrollments

- Display all the students enrolled in the enroll page.

6.13. Create a new function **getAllEnroll** to display all the enrollments in a table in the enroll page.

Here is the code.

```
function getAllEnroll(data) {
    const allenroll = JSON.parse(data)
    allenroll.forEach(enroll => {
        showTable(enroll)
    });
}
```

6.14. Include the API call in the function assigned to **window.onload** property.

```
    fetch('/enrolls')
    .then(response => response.text())
```

```
    .then(data => getAllEnroll(data));
```

4. Delete Enrollment

- Delete a student enrolled in a particular course.

6.15. In the enroll.js define a new function **deleteEnroll.** The function receives one parameter which is the object which triggered the function call.

Here is the code.

```
const deleteEnroll = async(r) => {
    if (confirm('Are you sure you want to DELETE this?')){
        selectedRow = r.parentElement.parentElement;
        sid = selectedRow.cells[0].innerHTML;
        cid = selectedRow.cells[1].innerHTML;
        fetch('/enroll/'+sid+"/"+cid, {
            method: "DELETE",
            headers: {"Content-type": "application/json; charset=UTF-8"}
        }). then( response => {
            if (response.ok) {
                var rowIndex = selectedRow.rowIndex; // index starts from 0
                if (rowIndex>0) { //th is row 0
                    document.getElementById("myTable").deleteRow(rowIndex);
                }
            }
        });
    }
}
```

That's it. Now your application is almost complete.

#Reminder push your source code to the GitLab repository.

7. Testing

Writing Unit Test in Golang

Testify Package

Go already has a standard test package, **testing**. Why would we need another library for writing tests?
Go standard test package lacks some useful features like **assertion**, **mocking,** and **test suite**. **Testify** is like an extension for the Go standard test package to do all those things; however, you need the **testing** package in order to use Testify.

Few tips for writing tests:

1. Keep your test files in the same directory as the function to be tested.
2. A test function name should begin with the word **Test**.

7.1. To start with, create a new test file, admin_test.go to test admin login functionality.

7.2. Use the command below to install the **assert** package which is defined in the **testify package.**

```
go get github.com/stretchr/testify/assert
```

7.3. In the admin_test.go define the test functions as shown below.

```go
package controller

import (
    "bytes"
    "io"
    "net/http"
    "testing"

    "github.com/stretchr/testify/assert"
)

// test case 1 for registered admin
func TestAdmLogin(t *testing.T) {
    // api endpoint to call
```

```
    url := "http://localhost:8080/login"
    // data of type byte slice
    var jsonStr = []byte(`{"email":"pc@gmail.com", "password":"pass"}`)
    // create http request
    req, _ := http.NewRequest("POST", url, bytes.NewBuffer(jsonStr))
    // set request header
    req.Header.Set("Content-Type", "application/json")
    // create a pointer variable client which points to Client struct
type
    client := &http.Client{}
    // client sends http request using Do() and gets http response
    resp, err := client.Do(req)
    // handle error if any
    if err != nil {
        panic(err)
    }
    // defer the closing of response body until function terminates
    defer resp.Body.Close()
    // get data from the response body. body is of type []byte
    body, _ := io.ReadAll(resp.Body)
    // validate if response status is same as expected status code
    assert.Equal(t, http.StatusOK, resp.StatusCode)
    expResp := `{"message":"login success"}`
    // validate if response body is same as expected response body
    assert.JSONEq(t, expResp, string(body))
}

// test case 2 for admin which does not exist
func TestAdmUserNotExist(t *testing.T) {
    url := "http://localhost:8080/login"
    var data = []byte(`{"email": "tsh@msitprogram.net", "password":
"pass1"}`)
    // create req object
    req, _ := http.NewRequest("POST", url, bytes.NewBuffer(data))
    req.Header.Set("Content-Type", "application/json")
    // create client
    client := &http.Client{}
    // send POST request
```

```
    resp, err := client.Do(req)
    if err != nil {
        panic(err)
    }
    defer resp.Body.Close()
    body, _ := io.ReadAll(resp.Body)
    assert.Equal(t, http.StatusUnauthorized, resp.StatusCode)
    assert.JSONEq(t, `{"error":"sql: no rows in result set"}`,
string(body))

}
```

- **t** is a pointer variable which points to type **T**. T is passed to test functions (test cases) as it provides methods to manage test state (test status) and support formatted test logs, such as **t.Errorf()**.

assert package

- The **Equal** function in the **assert** package (**assert.Equal)** is to **assert** the value of the actual and the expected result. The test passes if they are equal. Otherwise, the test fails.
- Every assert package function takes the **testing.T** object as the first argument. This is how it writes the errors out through the normal **go test** capabilities.
- Every assert function returns a **bool** indicating whether the assertion was successful or not, this is useful for if you want to go on making further assertions under certain conditions.

➢ Run the test and check if the test case is passed.

7.4. Create a new file **student_test.go** to define test cases to test API related to the student object.

student_test.go

```
package controller

import (
    "bytes"
    "io"
    "net/http"
```

```
    "testing"

    "github.com/stretchr/testify/assert"
)

func TestAddStudent(t *testing.T) {
    url := "http://localhost:8080/student"
    var jsonStr = []byte(`{"stdid":1002, "fname": "Sangay", "lname": "Lhamo",
"email":"sl@gmail.com"}`)
    req, _ := http.NewRequest("POST", url, bytes.NewBuffer(jsonStr))
    req.Header.Set("Content-Type", "application/json")

    client := &http.Client{}
    resp, err := client.Do(req)

    if err != nil {
        panic(err)
    }
    defer resp.Body.Close()

    body, _ := io.ReadAll(resp.Body)
    assert.Equal(t, http.StatusCreated, resp.StatusCode)
    expResp := `{"status":"student added"}`
    assert.JSONEq(t, expResp, string(body))

}

func TestGetStudent(t *testing.T) {
    c := http.Client{}
    r, _ := c.Get("http://localhost:8080/student/1002")
    body, _ := io.ReadAll(r.Body)
    assert.Equal(t, http.StatusOK, r.StatusCode)
    expResp := `{"stdid":1002, "fname":"Sangay", "lname":"Lhamo",
"email":"sl@gmail.com"}`
    assert.JSONEq(t, expResp, string(body))
}

func TestDeleteStudent(t *testing.T) {
```

```
    url := "http://localhost:8080/student/1002"
    req, _ := http.NewRequest("DELETE", url, nil)
    client := &http.Client{}
    resp, err := client.Do(req)
    if err != nil {
        panic(err)
    }
    defer resp.Body.Close()

    body, _ := io.ReadAll(resp.Body)
    assert.Equal(t, http.StatusOK, resp.StatusCode)
    expResp := `{"status":"deleted"}`
    assert.JSONEq(t, expResp, string(body))

}

func TestStudentNotFound(t *testing.T) {
    assert := assert.New(t)
    c := http.Client{}
    r, _ := c.Get("http://localhost:8080/student/1002")
    body, _ := io.ReadAll(r.Body)
    assert.Equal(http.StatusNotFound, r.StatusCode)
    expResp := `{"error":"Student not found"}`
    assert.JSONEq(expResp, string(body))
}
```

To do: You can create similar tests for the course and enroll.

#Reminder to push your source code to the GitLab repository.

8. Deployment

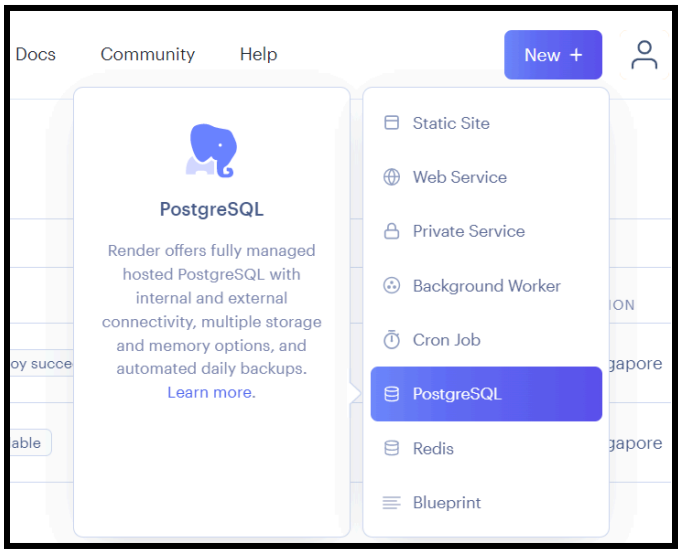
- **Render** is a cloud application hosting platform for developers, and they offer an array of services, such as **static sites**, **web services**, **database**, etc.
- The **free tier** instance type has 256 MB RAM, a shared CPU, and 1GB storage. Also, PostgreSQL is free for the first 90 days and will be deleted if not upgraded.

8.1. To start with login to render.com account. If you don't have an account, sign up first.

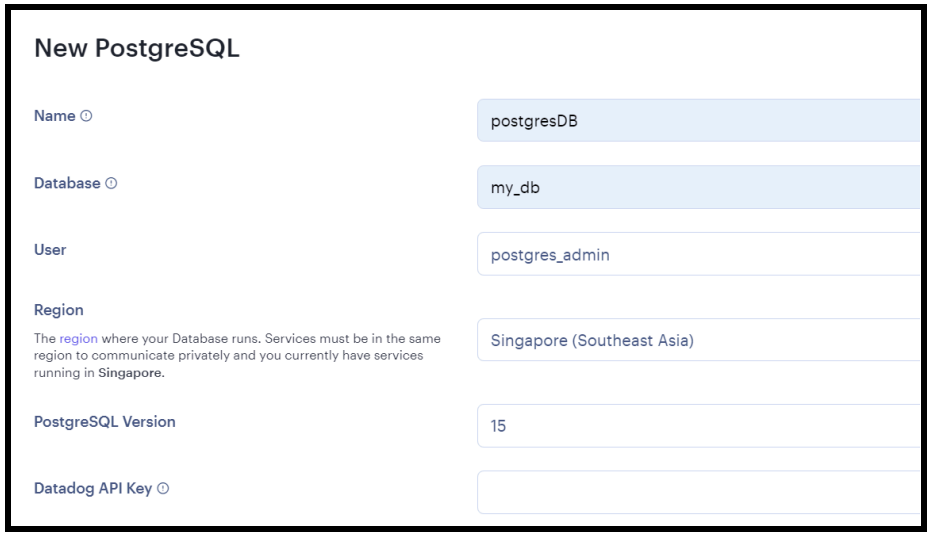
Creating PostgreSQL Database

➢ Before deploying the application first, set up the database service on the render platform.

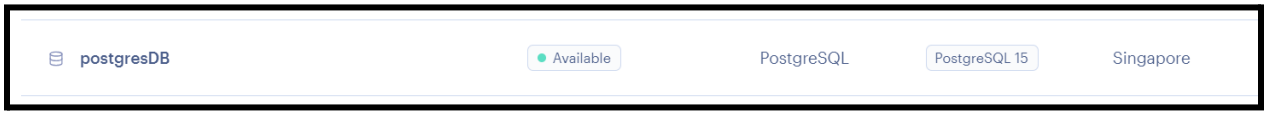
8.2. Click on the **New** menu on the right hand corner. Choose **PostgreSQL** from the drop down list item.



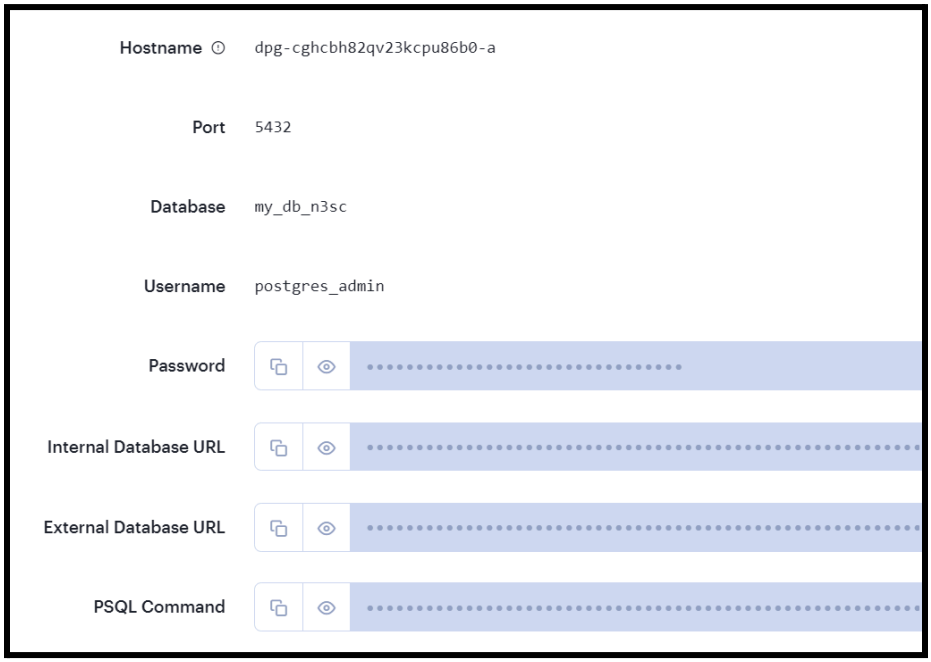
8.3. Fill in the necessary information like host name, database name, username. You can give any suitable names of your choice. Then click on the **Create Database** button at the bottom of the page.



➢ If the database is successfully created you will see in your dashboard having status as **Available**.



➢ Click on the database to see the details. You will see the following information.

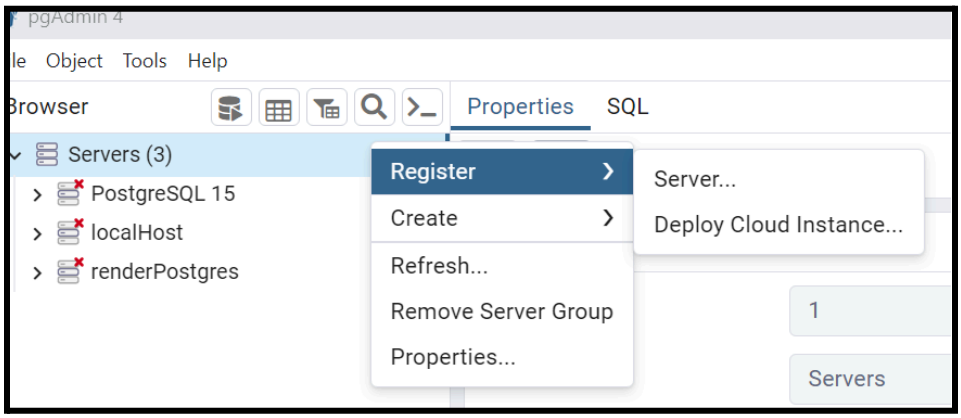


Connect to Render PostgreSQL using pgAdmin

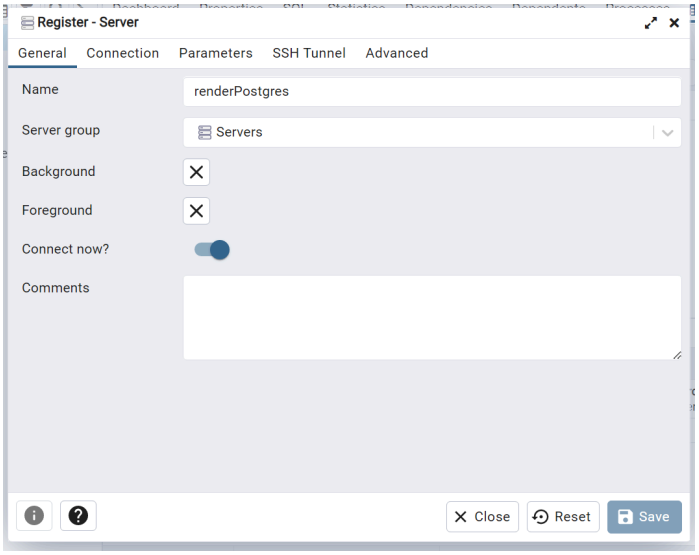
- Render lets you create databases, but not tables. You can use a tool like **pgAdmin** to create tables.
- pgAdmin is a GUI tool to interact with databases and perform any database administration task.

8.4. Go to the [pgAdmin download page] and select your operating system. Once it is installed, launch pgAdmin.

8.5. Register the database server. Right click on the Servers, choose Register from the options and choose server.



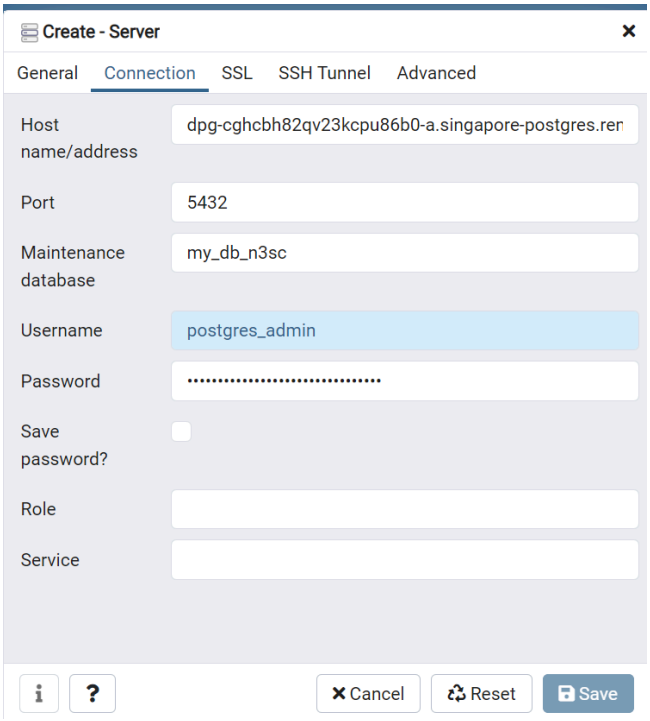
8.6. Provide the server name you want to keep.



8.7. Move to the Connection tab. Provide the host name, port number, database name (maintenance database), username and password. You will find this information in the postgreSQL created in render.

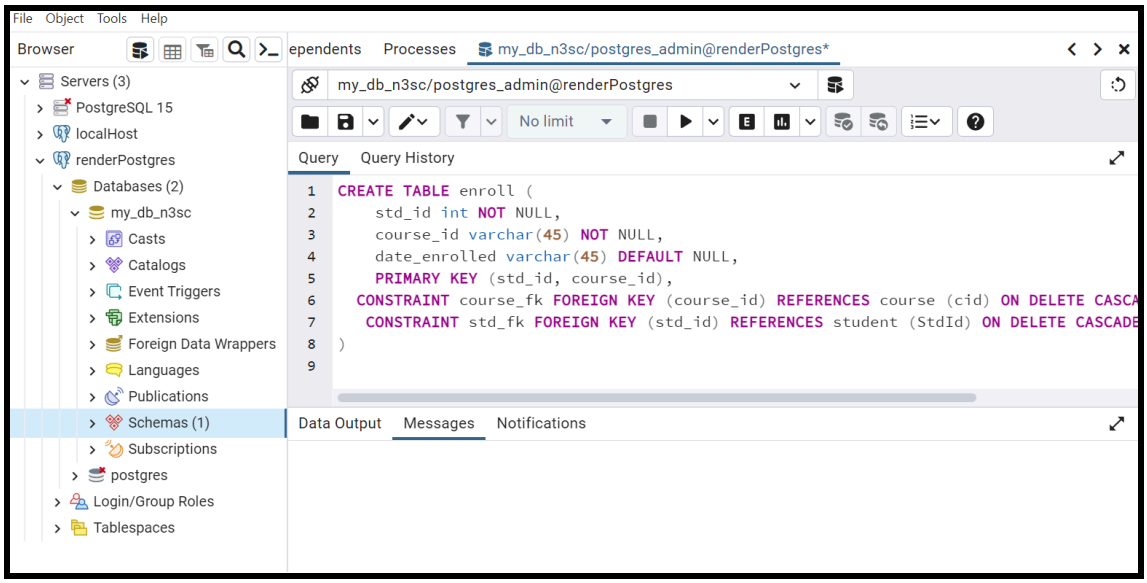
The **host name** comes from the **external** database URL. You take the string between the '@' and your database name. For example if the external database URL was "abc@def-a.oregon.postgres.com/database_name", then the host name is "def-a.oregon.postgres.com'.

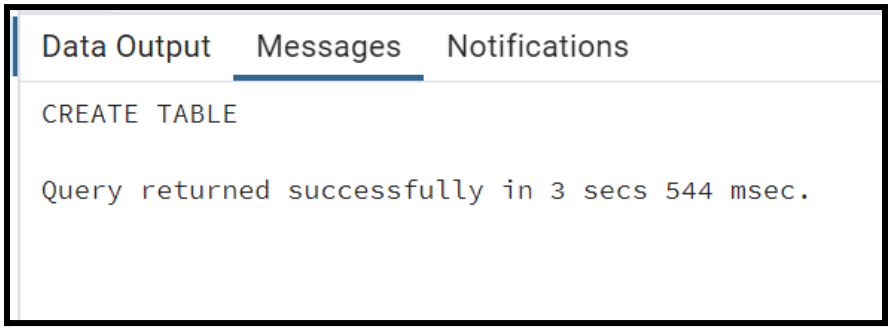
8.8. Then click on the **Save** button to set up the connection.



Creating Tables Using pgAdmin

8.9. Click on the server created (**renderPostgres** in this case). Click on the **Databases** in the option and click on your database name (**my_db_n3sc**). From the list right click on **Schemas** and select **Query Tool**. Paste your SQL query in the text area and press the **play** button on the top to execute the query.





8.10. Create all the remaining tables.

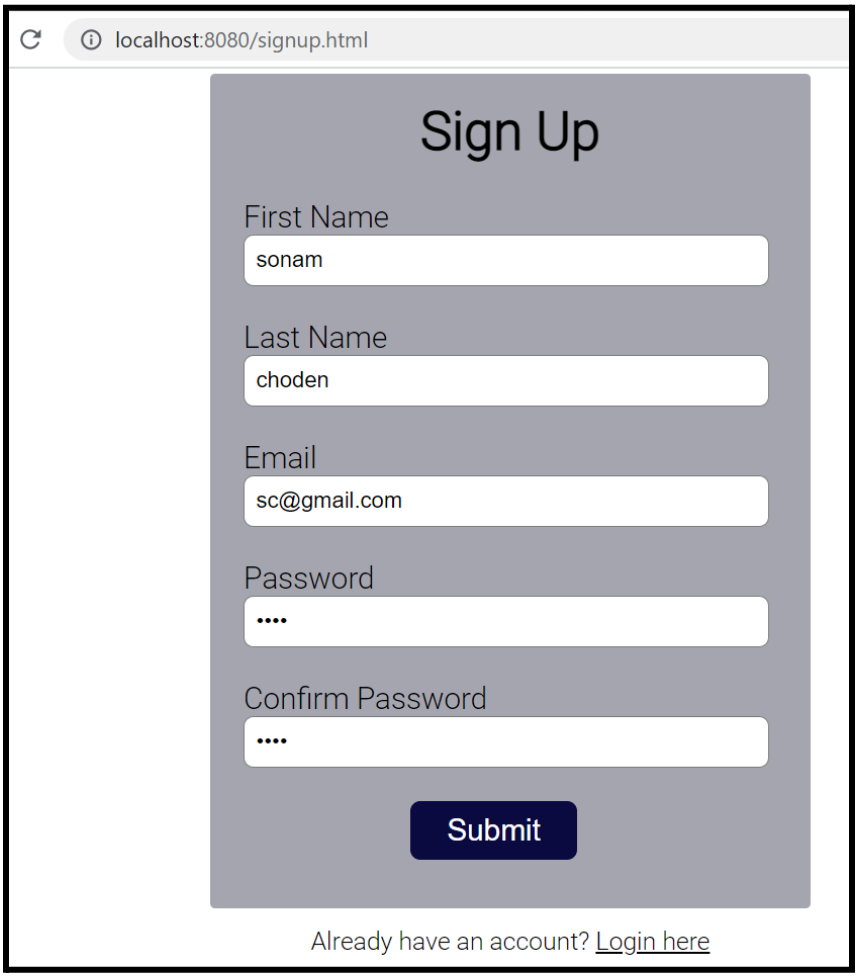
Now your render database instance is ready to store data.

Connecting Local Application to the Render PostgreSQL Database

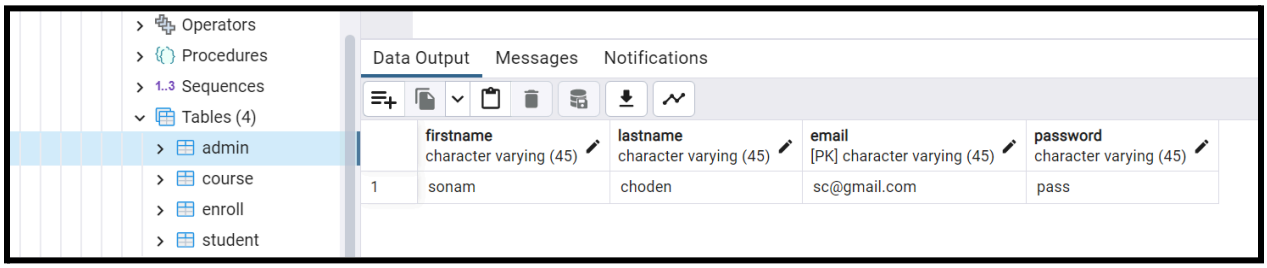
8.11. Now change the database details in the db.go file. Use the database information available in render postgreSQL.

```
const (
    postgres_host = "dpg-cghcbh82qv23kcpu86b0-a.singapore-postgres.render.com"
    postgres_port     = 5432
    postgres_user     = "postgres_admin"
    postgres_password = "NyoDGhU7eUApMbJrMxYIa0MaPDh5jDrh"
    postgres_dbname   = "my_db_n3sc"
)
```

8.12. Try running the application again and check if the application works the same way.
Sign up a new user and login using the credentials set. Check in the pgAdmin if the data is stored in the render postgres database.



8.13. Check the admin table if the admin data is saved.



➢ If you are not able to perform signup operation, remove the **sslmode** parameter from the database **connection string**.

- PostgreSQL has native support for using SSL connections to encrypt client/server communications using TLS protocols for increased security.

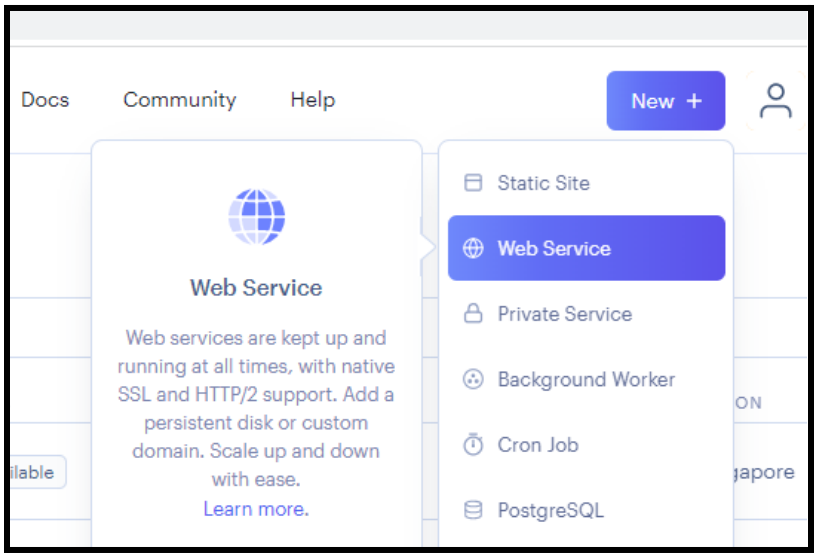
  Now you have successfully connected your application running on localhost to the database hosted on the render platform.

➢ Let's proceed to deploying the application on the render platform.

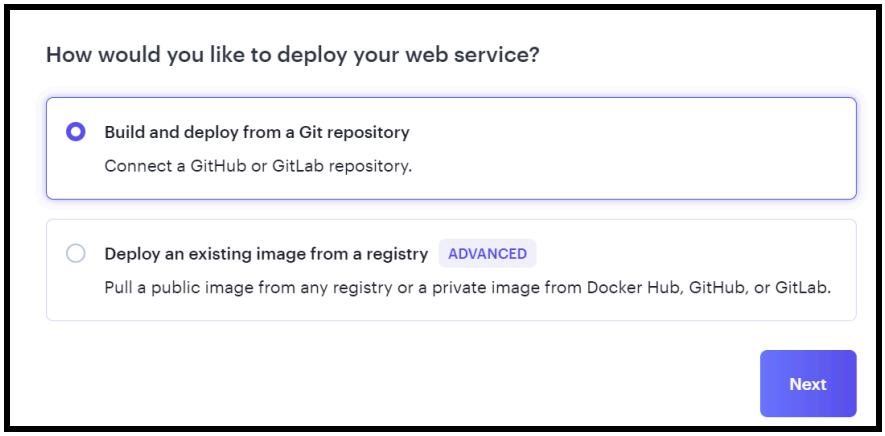
Deploying Web Service

8.14. Push your db.go file changes to the GitLab repository.

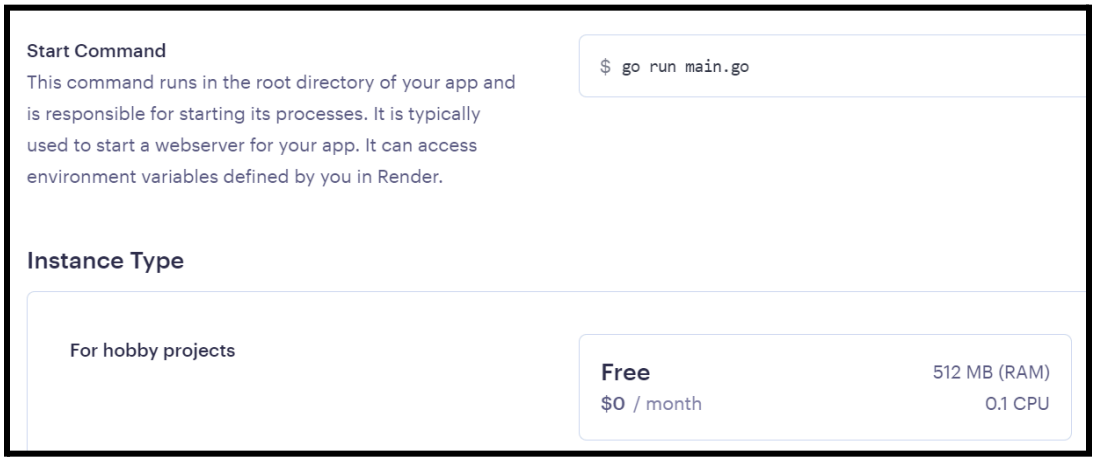
8.15. Click on the **New** blue button at the right hand corner. Select **Web Service** from the drop down menu.



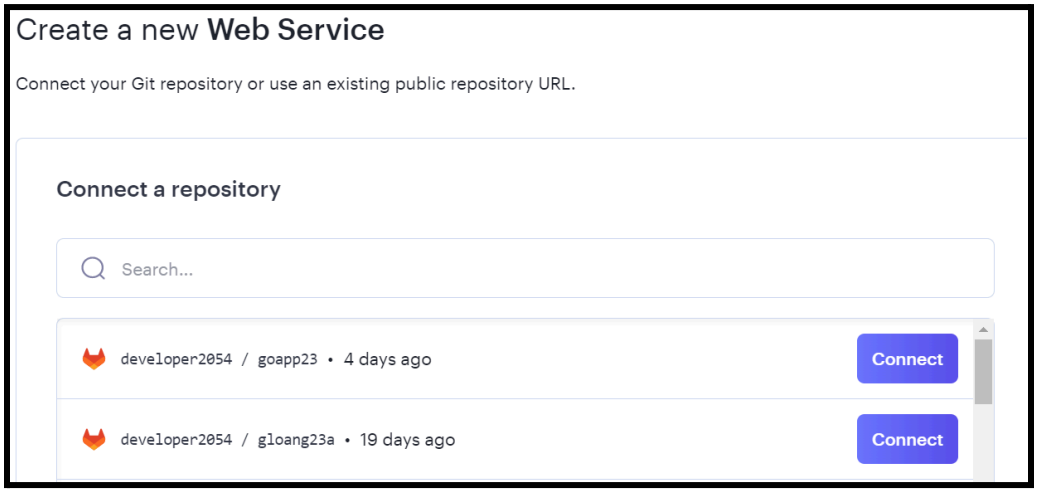
8.16. Select the first option to deploy by pulling the source code from the GitLab repository.



8.17. Put in the start command as **go run main.go.** And choose the **free** instance type.

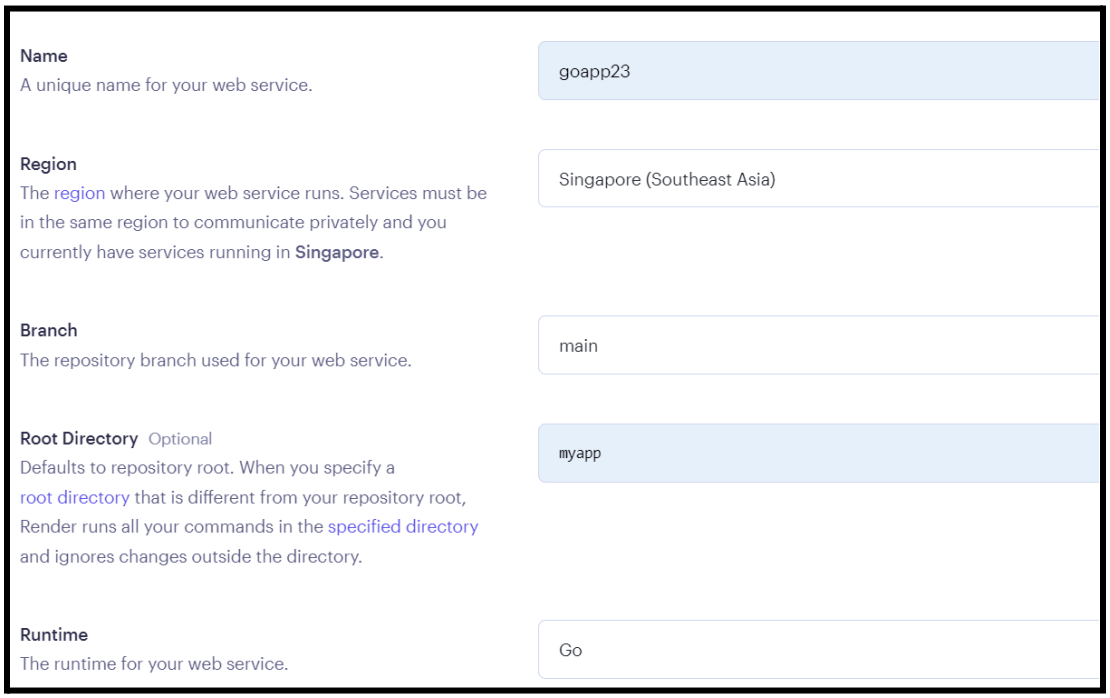


8.18. Choose the **GitLab repository** and click on **Connect** button to connect to the repository. Make sure you have linked the render account to your GitLab account.

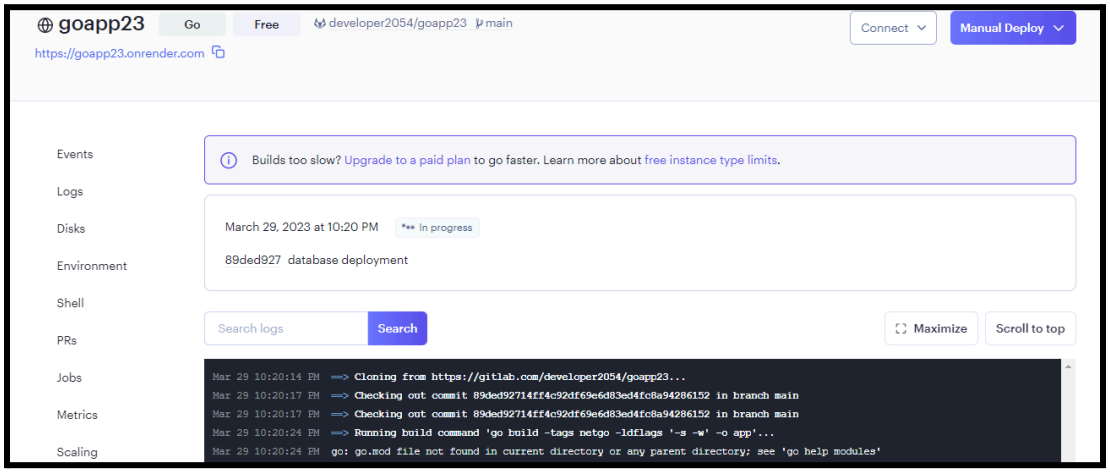


8.19. Fill in the necessary information to deploy a web service. Choose a **region** closer to your current location so that it reduces the latency. **Root directory** is the name of the directory where your **package main** is defined. Choose the **Go** runtime environment.

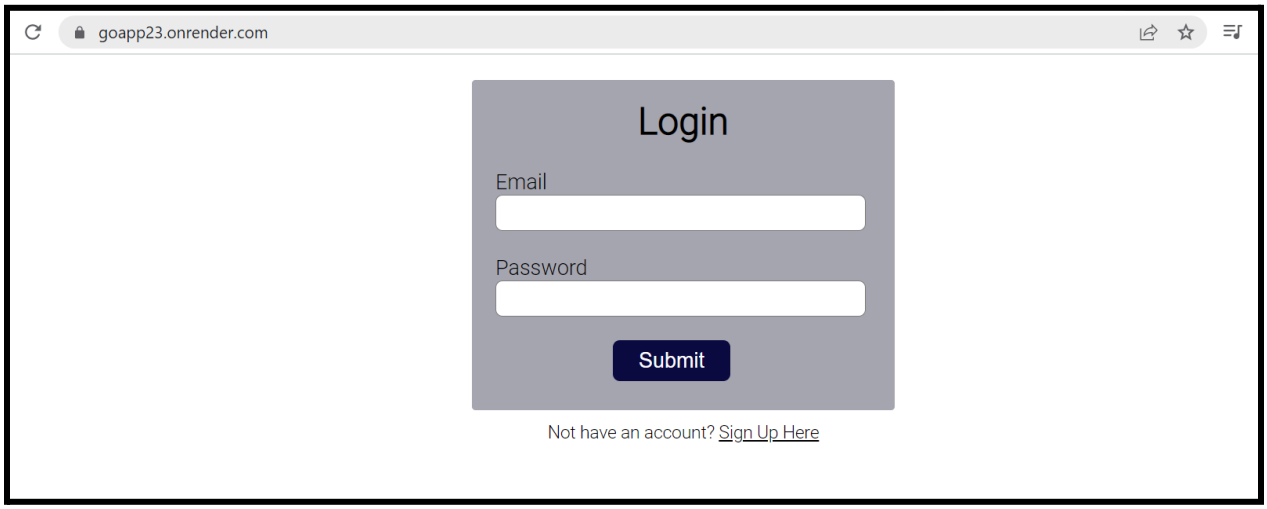
Then click on the **Create Web Service** button at the bottom of the page.



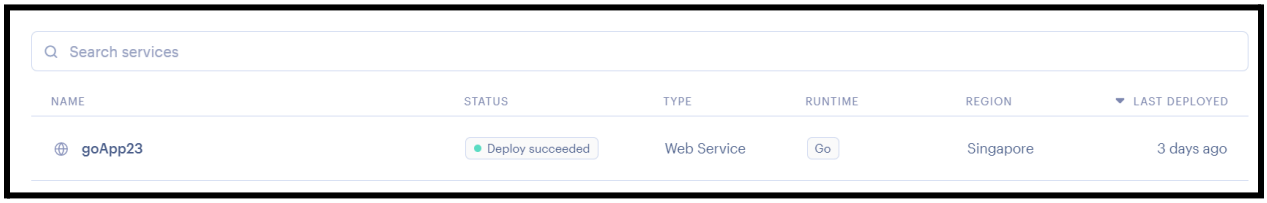
➢ You will see the deployment process log in the console.



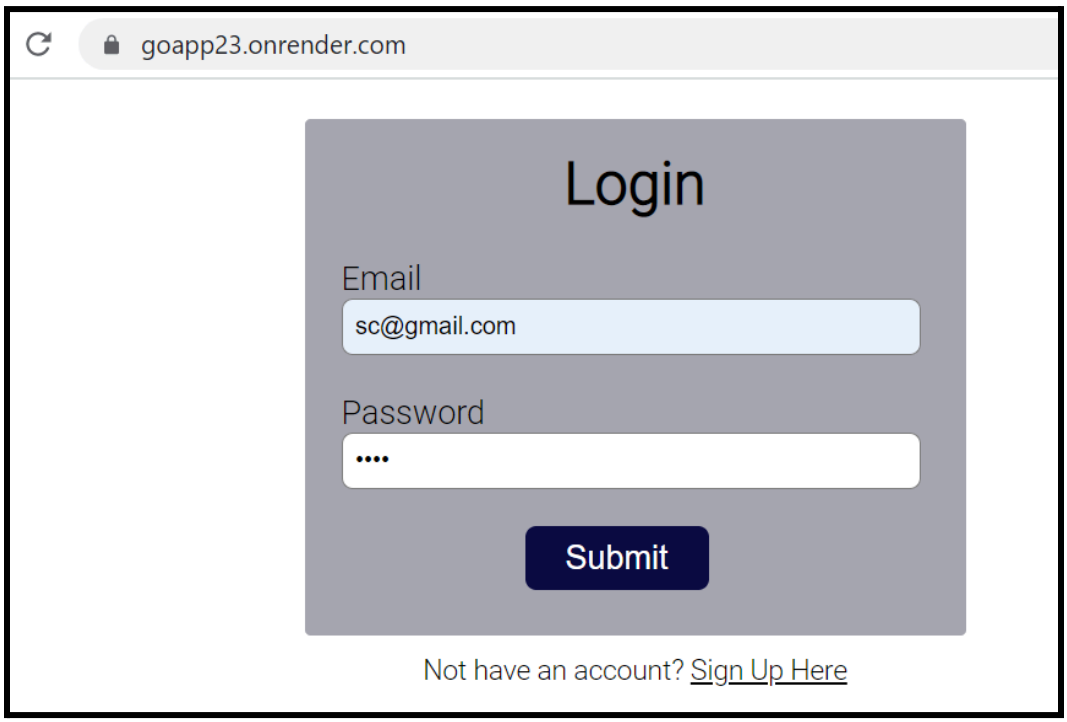
8.20. If the application deployment is successful, click on the URL at the top left corner to launch the application in the browser.



➢ You will see your **web service** listed in your render dashboard.



8.21. Login using the same credentials used above.



- Remember that when you make **changes** in your source code, **push** the changes to the remote repository. However, you do **not** need to create a new web service to **redeploy** the changes. Render will **automatically** redeploy the changes as and when the changes are committed to the remote repository.

- You can view the **deployment history** in the **Events** page.

***** THE END *****

Reference

https://towardsdatascience.com/everything-you-need-to-know-about-mvc-architecture-3c827930b4c1
https://medium.com/@sourabhbagrecha/how-to-handle-error-401-unauthorized-request-in-react-query-732297f24285
https://www.calhoun.io/connecting-to-a-postgresql-database-with-gos-database-sql-package/
https://www.sohamkamani.com/golang/session-cookie-authentication/
https://www.sohamkamani.com/golang/jwt-authentication/
https://medium.com/geekculture/how-to-create-and-connect-to-a-postgresql-database-with-render-and-pgadmin-577b326fd19d
https://www.cloudflare.com/learning/privacy/what-are-cookies/
https://medium.com/nerd-for-tech/writing-unit-tests-in-golang-part-1-introducing-testify-c0d458442412
https://medium.com/nerd-for-tech/writing-unit-tests-in-golang-part3-test-suite-6cca903be9ab